

Agentic Ontology of Work (AOW)

A semantic model for enterprise work performed by AI agents, people, and systems

Version 2.0 · September 2026 · Manish Garg, Skan.ai · Ontology DOI:
<https://doi.org/10.5281/zenodo.22945905>

Preprint: <https://doi.org/10.5281/zenodo.22945754>

Licensed [CC-BY 4.0](#). The machine-readable ontology, shapes, schemas, and examples that accompany this paper are licensed Apache 2.0 and published at github.com/Skandotai/agentic-ontology-of-work.

Abstract

Terms such as "agent," "task," "workflow," and "orchestration" are used inconsistently across vendors and teams. As a result, agentic systems are difficult to integrate, policies are applied inconsistently, and audit trails end where work passes from one system to another.

The Agentic Ontology of Work (AOW) is a platform-agnostic semantic model for enterprise work performed by AI agents, people, and systems. It defines 25 classes and 33 relationships. The model links each business Objective to the Intents, Plans, and Tasks derived from it, the Actions that Actors (Agents and people) take using Skills, the Results those Actions produce, and the Outcomes those Results contribute to. It governs autonomy through Policy, Confidence, a five-level Assurance Level scale, an independent Guardian, and Feedback recorded in Memory.

Version 2.0 defines Actions, Tasks, and autonomy levels, which version 1.0 named but did not define, and adds entities for the people who participate in the work. It also publishes the ontology in OWL, JSON-LD, SHACL, and JSON Schema, with validated examples and queries. The queries confirm that every Result can be traced to its Objective and that no Agent acted without the oversight its Assurance Level requires.

1 Introduction

1.1 The rise of the agentic enterprise

Enterprises are moving from systems that automate tasks to systems that interpret goals, reason, and act across complex operations. Contributing developments include multi-agent systems, learned behavior, contextual awareness, structured governance, continuous feedback loops, and observability at enterprise scale.

Enterprises do not have an agreed way to describe what these systems are, how they behave, how they relate to each other and to the people they work with, and how they must be governed.

Service-Oriented Architecture (SOA) addressed a similar problem. It gave the industry a widely adopted formalism for services, endpoints, and contracts, and its shared vocabulary was a large part of its adoption. Agentic systems need an equivalent common description of Intents, Agents, Skills, Policies, Context, Outcomes, Assurance, Memory, and Feedback.

1.2 Why an ontology, not a glossary

A glossary defines words. An ontology is "an explicit specification of a conceptualization" (Gruber, 1993): it names the kinds of things in a domain, the attributes those things have, and the relationships that connect them, precisely enough that software can check whether a description conforms.

Without a shared ontology, enterprises face:

- inconsistent definitions, in which "agent," "workflow," and "task" overlap
- brittle integrations between platforms that model the same work differently
- governance blind spots where no one can say which rules applied to an action
- ambiguous responsibility when work passes between agents and people
- no reliable way to scale autonomous activity

A shared ontology supports governance and compliance, makes behavior explainable, improves interoperability across platforms, and reduces ambiguity.

1.3 Goals

AOW aims to:

- establish a platform-agnostic semantic framework for agentic work

- enable interoperability across multi-agent systems, robotic process automation, AI systems, and human workflows
- support governance, auditability, and risk management by design
- provide a foundation that standards bodies and reference architectures can build on
- help enterprises scale intelligent automation safely, with trust and transparency

1.4 Scope and non-goals

AOW describes work: what is being attempted, by whom, under what constraints, with what effect, and what was learned. It does not specify:

- **how agents are built.** Models, prompts, and frameworks are implementation details. An Agent's `implementation_ref` may point to them.
- **how agents talk to each other.** Protocols such as Agent2Agent (A2A) and the Model Context Protocol (MCP) carry AOW entities; AOW does not redefine them (section 12.6).
- **a policy language.** A Policy's rules may be written in ODRL, Rego, Cedar, DMN, or plain text (section 4).
- **a storage technology.** AOW can be stored in a knowledge graph, a relational database, or a document store (section 12.4).

1.5 What changed from version 1.0

Version 1.0 (January 2026) set out the four-layer model, fifteen canonical entities, the lifecycle, and the validation criteria. Version 2.0 keeps all of that and is backward compatible in meaning: every 1.0 entity still exists and still means what it meant. The main changes are:

- **Eight new classes** fill gaps 1.0 named but did not define: Work Item, Signal, and Observation (Perception); Plan and Task; and Actor, Human Actor, and Role, which make people first-class participants. Orchestrator and Guardian become kinds of Actor.
- **Action is now defined.** 1.0 listed it in the glossary and registry but gave it no section, and said in one place that Skills produce Results and in another that Actions do. In 2.0, a Skill is a capability, an Action is its use, and the Action produces the Result.
- **The Assurance Level scale is defined**, from AL0 (Manual) to AL4 (Autonomous), with a rule for the effective level when an Agent, a Task, and a Skill disagree (section 7).
- **Provenance and Versioning become a facet** of every entity, expressed with W3C PROV-O, rather than two entities of their own (section 5).

- **Every entity has attributes and relationships.** In 1.0, Guardian, Memory, Provenance, and Assurance Level had none, or only one of the two.
- **The ontology ships in machine-readable form**, with validation (section 9).
- **Editorial fixes:** consistent numbering, corrected example schemas, and a more measured claim about what the ontology is.

Appendix B lists every change.

2 Design principles

AOW is built on seven principles. The first six are carried over from version 1.0; the seventh is new.

Principle 1: Platform agnosticism. The ontology describes concepts, not implementations. No vendor-specific models, training mechanisms, or product metaphors are embedded in it.

Principle 2: Hierarchical simplicity with semantic completeness. The ontology must be rich enough to model complex work and simple enough for architects, engineers, risk teams, and business owners to use.

Principle 3: Governed autonomy. Agentic systems operate under constraints, not unchecked freedom. Policy, Assurance Levels, and provenance are first-class.

Principle 4: Observability and explainability. No action happens without visibility, and no decision is made without a traceable rationale.

Principle 5: Contextual intelligence. Actions are always understood in relation to the environment, history, constraints, and intended Outcomes.

Principle 6: Continuous learning. The ontology supports the accumulation of experience and adaptation as environments shift and agents evolve.

Principle 7: Actor neutrality. Work is described the same way whether an Agent or a person performs it. A Task can be assigned to either. An Action has the same attributes and is recorded the same way in both cases. Work can therefore move between people and agents without being remodeled, and human oversight is recorded in the same form as agent activity.

3 The four-layer stack

AOW models work as a graph connecting Objectives, Intents, Context, constraints, Actors, Actions, Observations, Feedback, and Outcomes. The graph is organized into four layers.

Layer	Purpose	Entities
1 Perception	Records the work in progress and what is observed about it	Work Item, Signal, Observation
2 Cognition	Defines what should happen and the limits that apply	Objective, Intent, Context, Policy, Plan
3 Execution	Records who performed the work and what it changed	Task, Actor, Agent, Human Actor, Orchestrator, Role, Skill, Action, Result
4 Assurance	Governs autonomy, measures outcomes, and records feedback	Confidence, Assurance Level, Guardian, Outcome, Feedback, Memory, Operational Memory, Knowledge Base

Layer 1: Perception records the Work Items in progress and the Signals (events, logs, documents, interactions, state changes, metrics) and Observations about them. Version 1.0 named Telemetry Events, Signals, Session State, and Environmental Observations here without defining them. In 2.0, a telemetry event is a Signal, session state is session Context, and an environmental observation is an Observation.

Layer 2: Cognition evaluates goals, constraints, context, and policies to decide what should happen next, and plans how.

Layer 3: Execution is where Actors take on Tasks, invoke Skills through Actions, and produce Results.

Layer 4: Assurance sets permitted autonomy, measures confidence, oversees behavior, evaluates Outcomes, and records Feedback in Memory.

The layers describe what an entity is for, not a deployment architecture. One system may span several layers, and entities in every layer carry the provenance facet described in section 5.

4 Canonical entities

This section defines each entity: what it is, why it exists, its attributes, and the relationships it can start. Attribute and relationship keys are shown as they appear in AOW JSON documents (`snake_case`); the RDF property for each is the same name in `lowerCamelCase` in the `aow: namespace` (`goal_statement` becomes `aow:goalStatement`). Every entity also has an `id` and a `type` , and may carry the common attributes `label` , `description` , and the provenance facet (section 5).

Where a relationship is shown with more than one target, any of them is allowed. "Required" means a conformant document must include it.

4.1 Work Item

Layer: Perception · since 2.0 · `aow:WorkItem`

Definition. The unit of business work that the rest of the ontology is about: a claim, an application, an order, a case, a ticket.

Purpose. Identifies the item that Intents, Tasks, Actions, and Observations refer to, so work on one item can be followed across systems, agents, and people.

Attribute	Type	Required	Description
<code>item_type</code>	string	yes	The kind of work item, for example "insurance-claim" or "purchase-order".
<code>external_ref</code>	string		Identifier of the item in its system of record.
<code>state</code>	string		Current business state of the item, in the vocabulary of its system of record.
<code>opened_at</code>	datetime		When the item entered the process.
<code>closed_at</code>	datetime		When the item left the process, if it has.

Notes.

- In version 1.0 the work item was implicit in Context (for example, "ctx-claim-84933"). As a separate entity, it can be referenced by several Intents and Tasks.
- Corresponds to a case or object in process mining (IEEE 1849 XES and the OCEL 2.0 object-centric event log format).

4.2 Signal

Layer: Perception · since 2.0 · aow:Signal

Definition. A raw, time-stamped trace emitted by a system, a person, or the environment: an event, a log line, a user-interface interaction, a document arriving, a state change, a metric sample.

Purpose. Records what happened before any interpretation. Observations, Context, and audit trails are derived from Signals.

Attribute	Type	Required	Description
signal_type	one of event , log , metric , interaction , document , state-change	yes	The kind of trace.
source	string	yes	The system, sensor, application, or channel that emitted the Signal.
observed_at	datetime	yes	When the Signal occurred.
payload_ref	uri		Where the raw payload is stored, if it is kept.
sensitivity_level	one of public , internal , confidential , restricted		Handling classification of the payload.

Relationships.

- generated_by → Action

Notes.

- Version 1.0 listed Telemetry Events, Signals, Session State, and Environmental Observations as Perception-layer elements without defining them. In 2.0 a Telemetry Event is a Signal whose signal_type is "event" or "metric"; Session State is Context with context_type "session".

- A Signal can be linked to the Action that generated it (`generated_by`). Agent activity can then be analyzed with the same tools used for human and system activity.

4.3 Observation

Layer: Perception · since 2.0 · aow:Observation

Definition. An interpreted fact about a Work Item, Actor, or Action, derived from one or more Signals and carrying a Confidence.

Purpose. Separates recorded evidence (Signal) from its interpretation (Observation), so an incorrect interpretation can be identified and corrected.

Attribute	Type	Required	Description
<code>observed_property</code>	string	yes	What was observed, for example "claim_amount" or "handling_time".
<code>observed_value</code>	json		The observed value, in any JSON form.
<code>observed_at</code>	datetime		When the observation was made.
<code>method</code>	string		How the value was derived, for example "document-extraction" or "rule".

Relationships.

- `generated_by` → Action
- `derived_from` → Signal (one or more) (*required*)
- `about` → Work Item / Actor / Action / Task
- `has_confidence` → Confidence

Notes.

- "The claim amount is \$38,200" is an Observation; the scanned claim form it was extracted from is a Signal.

4.4 Objective

Layer: Cognition · since 1.0 · aow:Objective

Definition. A business-level goal expressed in strategic or operational terms.

Purpose. Connects agentic activity to specific, measurable enterprise value.

Attribute	Type	Required	Description
kpi_target	string		The measurable target, for example "claims cycle time -20%".
priority	one of low , medium , high , critical		Relative business priority.
time_horizon	string		The period over which the Objective is pursued, for example "FY2027".
description	text	yes	Longer human-readable explanation.

Notes.

- Objective answers why. Intent answers what.

4.5 Intent

Layer: Cognition · since 1.0 · aow:Intent

Definition. A structured, actionable goal derived from an Objective and interpretable by agents, orchestrators, and people.

Purpose. States a business goal in a form that can be planned, assigned, governed, and verified.

Attribute	Type	Required	Description
<code>goal_statement</code>	text	yes	What the Intent is meant to achieve, stated as an outcome rather than a procedure.
<code>constraints</code>	string-list		Free-text constraints that are not (yet) captured as Policies.
<code>priority</code>	one of <code>low</code> , <code>medium</code> , <code>high</code> , <code>critical</code>		Relative priority.
<code>acceptable_risk_band</code>	one of <code>low</code> , <code>medium</code> , <code>high</code>		The level of residual risk the requester accepts.
<code>status</code>	one of <code>proposed</code> , <code>active</code> , <code>fulfilled</code> , <code>abandoned</code>		Where the Intent is in its life.

Relationships.

- `serves_objective` → Objective (*required*)
- `shaped_by` → Context (one or more)
- `concerns` → Work Item (one or more)
- `constrained_by` → Policy (one or more)
- `requires_assurance` → Assurance Level (*required*)

Notes.

- Objective answers why. Intent answers what. Plan answers how.

4.6 Context

Layer: Cognition · since 1.0 · `aow:Context`

Definition. The structured situational information that gives relevance to an Intent.

Purpose. Allows the same Intent to be handled differently in different situations, and records the information available to an Actor when it acted.

Attribute	Type	Required	Description
context_type	one of environmental , procedural , data , historical , session		The kind of situational information.
variables	json		The situational values themselves.
sensitivity_level	one of public , internal , confidential , restricted		Handling classification.
valid_until	datetime		When the Context should be considered stale.

Relationships.

- draws_on → Memory / Observation (one or more)

Notes.

- Context types: environmental, procedural, data, historical, session.

4.7 Policy

Layer: Cognition · since 1.0 · aow:Policy

Definition. Declarative constraints defining permissible, required, or restricted behavior.

Purpose. States governance rules in a form that can be checked by software. Policies constrain behavior. They do not perform Actions.

Attribute	Type	Required	Description
<code>rule_set</code>	text	yes	The rules, or a reference to them.
<code>rule_language</code>	string		The language <code>rule_set</code> is written in, for example "ODRL", "Rego", "DMN", "text".
<code>effect</code>	one of <code>permit</code> , <code>prohibit</code> , <code>require</code> , <code>escalate</code>		What the Policy does when it applies.
<code>trigger_conditions</code>	string-list		Conditions under which the Policy applies.
<code>severity</code>	one of <code>low</code> , <code>medium</code> , <code>high</code> , <code>critical</code>		Consequence of a violation.
<code>override_protocol</code>	text		Who may override the Policy, and how.

Relationships.

- `escalates_to` → Role / Human Actor (one or more)

Notes.

- AOW does not prescribe a policy language. `rule_set` may hold rules in ODRL, Rego, Cedar, DMN, or plain text, named in `rule_language`.

4.8 Plan

Layer: Cognition · since 2.0 · `aow:Plan`

Definition. An ordered or partially ordered set of Tasks intended to fulfill an Intent, produced by an Orchestrator or other Actor.

Purpose. Separates planning from execution, so plans can be reviewed, approved, and compared.

Attribute	Type	Required	Description
<code>status</code>	one of <code>draft</code> , <code>approved</code> , <code>executing</code> , <code>completed</code> , <code>failed</code> , <code>abandoned</code>		Where the Plan is in its life.
<code>exception_protocol</code>	text		What to do when a Task fails or a Policy blocks progress.

Relationships.

- `constrained_by` → Policy (one or more)
- `realizes` → Intent (*required*)
- `planned_by` → Actor
- `has_task` → Task (one or more)
- `approved_by` → Human Actor (one or more)
- `has_confidence` → Confidence

Notes.

- Listed as an optional concept in version 1.0; a full entity in 2.0.
- The Orchestrator's `dependency_graph` in version 1.0 is the Plan's set of Tasks and their `depends_on` links.

4.9 Task

Layer: Execution · since 2.0 · aow:Task

Definition. A unit of assigned work within a Plan, performed by an Actor filling a Role.

Purpose. Defines the unit of work that is assigned. A Task is assigned to an Actor. An Action is performed to complete it. A Skill is the capability the Action uses.

Attribute	Type	Required	Description
<code>status</code>	one of <code>submitted</code> , <code>working</code> , <code>input-required</code> , <code>completed</code> , <code>failed</code> , <code>canceled</code> , <code>rejected</code>		Where the Task is in its life.
<code>due_by</code>	datetime		When the Task must be completed.
<code>description</code>	text	yes	Longer human-readable explanation.

Relationships.

- `shaped_by` → Context (one or more)
- `concerns` → Work Item (one or more)
- `constrained_by` → Policy (one or more)
- `requires_assurance` → Assurance Level
- `depends_on` → Task (one or more)
- `assigned_to` → Actor
- `requires_role` → Role

Notes.

- Task states match the task states of the Agent2Agent (A2A) protocol, so Tasks can be exchanged over A2A without conversion.

4.10 Actor

Layer: Execution · abstract · since 2.0 · `aow:Actor`

Definition. Anything that can be assigned Tasks, perform Actions, and be held to account for them: an Agent, a person, an Orchestrator, or a Guardian.

Purpose. Allows work to be described the same way whether a person or an agent performs it.

Attribute	Type	Required	Description
name	string		Display name.

Relationships.

- constrained_by → Policy (one or more)
- fills_role → Role (one or more)
- has_skill → Skill (one or more)
- writes_to → Memory (one or more)

4.11 Agent

Layer: Execution · a kind of Actor · since 1.0 · aow:Agent

Definition. A software Actor that interprets Intents and Tasks and acts on them using Skills, within the autonomy it has been cleared for.

Purpose. Describes an agent by its permitted capabilities and autonomy, not by its implementation.

Attribute	Type	Required	Description
autonomy_level	→ AssuranceLevel		The highest Assurance Level the Agent has been cleared to operate at. The Agent may operate at a lower level on a given Task, never a higher one.
implementation_ref	uri		Pointer to the Agent's implementation, model card, or A2A Agent Card.

Notes.

- AOW does not describe models, prompts, or frameworks. implementation_ref may reference them.

4.12 Human Actor

Layer: Execution · a kind of Actor · since 2.0 · aow:HumanActor

Definition. A person who performs, approves, reviews, or oversees work.

Purpose. Records human participation, including approvals and decisions, in the same form as agent activity.

Attribute	Type	Required	Description
<code>organizational_unit</code>	string		The team or department the person works in.

4.13 Orchestrator

Layer: Execution · a kind of Actor · since 1.0 · `aow:Orchestrator`

Definition. A coordinating Actor that turns Intents into Plans, assigns Tasks to Actors, and manages sequencing and exceptions.

Purpose. Decomposes Intents into Tasks and coordinates their execution.

Attribute	Type	Required	Description
<code>planning_model</code>	string		How the Orchestrator plans, for example "rule-based", "LLM-planned", "hybrid".
<code>exception_protocol</code>	text		Default handling for failures across the Plans it runs.
<code>resource_constraints</code>	text		Limits on concurrency, cost, or time it must respect.

Notes.

- An Orchestrator may be an Agent, a workflow engine, or a person.

4.14 Role

Layer: Execution · since 2.0 · `aow:Role`

Definition. A named set of responsibilities and authority that an Actor can fill, such as "claims adjuster" or "eligibility reviewer".

Purpose. Allows Tasks, Policies, and escalations to target a responsibility instead of a specific person or agent.

Attribute	Type	Required	Description
<code>responsibilities</code>	string-list		What an Actor in this Role is accountable for.
<code>label</code>	string	yes	Short human-readable name.

Relationships.

- `constrained_by` → Policy (one or more)

4.15 Skill

Layer: Execution · since 1.0 · `aow:Skill`

Definition. A reusable capability that an Actor can invoke.

Purpose. Describes a capability's inputs, outputs, cost, and side effects, so it can be discovered, governed, and replaced.

Attribute	Type	Required	Description
<code>input_schema</code>	json		JSON Schema (or equivalent) for the Skill's inputs.
<code>output_schema</code>	json		JSON Schema (or equivalent) for the Skill's outputs.
<code>binding</code>	string		How to invoke it, for example an MCP tool name, an API endpoint, or a UI automation.
<code>side_effects</code>	one of <code>none</code> , <code>reversible</code> , <code>irreversible</code>		Whether invoking the Skill changes external state, and whether that can be undone.
<code>latency</code>	string		Typical or maximum latency, for example "< 1s".
<code>cost_profile</code>	one of <code>low</code> , <code>medium</code> , <code>high</code>		Relative cost per invocation.

Relationships.

- `constrained_by` → Policy (one or more)
- `requires_assurance` → Assurance Level

Notes.

- `side_effects` is new in 2.0. It records whether a Skill changes external state and whether the change can be reversed. Assurance Levels AL3 and AL4 do not permit irreversible Skills without approval.

4.16 Action

Layer: Execution · since 1.0 · `aow:Action`

Definition. A single execution event in which an Actor invokes a Skill, in service of a Task, at a point in time.

Purpose. Records who did what, when, and why. Every Result is linked to the Action that produced it.

Attribute	Type	Required	Description
<code>started_at</code>	datetime	yes	When the Action began.
<code>ended_at</code>	datetime		When the Action ended.
<code>status</code>	one of <code>succeeded</code> , <code>failed</code> , <code>blocked</code> , <code>escalated</code> , <code>compensated</code>		How the Action ended.
<code>rationale</code>	text		The Actor's stated reason for the Action, recorded for explainability.
<code>idempotency_key</code>	string		Key that makes a retried Action safe to repeat.
<code>inputs</code>	json		The inputs passed to the Skill.

Relationships.

- `concerns` → Work Item (one or more)

- `performed_by` → Actor (*required*)
- `fulfills` → Task
- `invokes` → Skill
- `approved_by` → Human Actor (one or more)
- `has_confidence` → Confidence

Notes.

- Named but not defined in version 1.0. In 2.0, the Action produces the Result. A Skill is a capability. An Action is one use of it.

4.17 Result

Layer: Execution · since 1.0 · `aow:Result`

Definition. The immediate, atomic effect or output of an Action.

Purpose. Records what an Action changed or produced, so it can be verified and aggregated into Outcomes.

Attribute	Type	Required	Description
<code>status</code>	one of <code>success</code> , <code>partial</code> , <code>failure</code>		Whether the Action achieved what it set out to.
<code>summary</code>	text		What the Result is, in words.
<code>artifacts</code>	uri-list		Documents, records, or state changes the Result produced.

Relationships.

- `approved_by` → Human Actor (one or more)
- `generated_by` → Action (*required*)
- `affects` → Work Item (one or more)
- `has_confidence` → Confidence
- `contributes_to` → Outcome (one or more)

Notes.

- A Result is local and immediate. An Outcome is aggregated and business-level.

4.18 Guardian

Layer: Assurance · a kind of Actor · since 1.0 · aow:Guardian

Definition. The oversight Actor that enforces Policy and Assurance Levels, monitors behavior, and escalates to people when needed.

Purpose. Separates performing work from checking it.

Attribute	Type	Required	Description
enforcement_mode	one of preventive , detective , corrective		Whether it blocks before, detects after, or repairs after the fact.
scope	text		The Actors, Skills, or Work Items it watches.

Relationships.

- enforces → Policy / Assurance Level (one or more)
- oversees → Actor / Plan / Skill (one or more)
- escalates_to → Role / Human Actor (one or more)

Notes.

- A Guardian may be software, a person, or both. Its interventions are recorded as Actions.

4.19 Confidence

Layer: Assurance · since 1.0 · aow:Confidence

Definition. A model-, rule-, or human-derived measure of certainty about an Observation, a Result, or an Action.

Purpose. Allows autonomy to depend on certainty. An Agent acts without approval only when its Confidence meets the threshold for its Assurance Level.

Attribute	Type	Required	Description
value	number (0–1)	yes	Certainty between 0 and 1.
source	one of model , heuristic , rule , ensemble , human		What produced the measure.
calibrated	boolean		Whether the value has been calibrated against observed accuracy.
volatility	one of low , medium , high		How much the value is expected to move as evidence arrives.

Relationships.

- compared_against → Assurance Level

4.20 Assurance Level

Layer: Assurance · since 1.0 · aow:AssuranceLevel

Definition. A governance-set level of permitted autonomy, with the human oversight and evidence it requires.

Purpose. Specifies the permitted autonomy and required human oversight for a piece of work. AOW defines five levels, AL0 to AL4.

Attribute	Type	Required	Description
<code>level</code>	integer (0–4)	yes	Position on the AL0 to AL4 scale.
<code>human_role</code>	text		What people do at this level.
<code>agent_role</code>	text		What Agents may do at this level.
<code>review_protocol</code>	text		The human oversight this level requires.
<code>default_confidence_floor</code>	number (0–1)		Confidence below which an Actor operating at this level must escalate, unless a Policy sets a different threshold.
<code>risk_class</code>	string		The class of risk this level is suited to.

Notes.

- Higher levels permit more autonomy and require more evidence before they are granted.

4.21 Outcome

Layer: Assurance · since 1.0 · `aow:Outcome`

Definition. The aggregated business impact of one or more Results, measured against an Objective.

Purpose. Measures the effect of work against the Objective it served.

Attribute	Type	Required	Description
kpi_impact	json		The measured change, for example {"cycle_time_hours": {"before": 48, "after": 0.12}}.
measured_at	datetime		When the Outcome was measured.
measurement_window	string		The period the measurement covers.
time_to_value	string		How long it took for the impact to appear.
risk_profile_change	text		How the risk picture changed as a result.

Relationships.

- evaluates → Objective (*required*)

4.22 Feedback

Layer: Assurance · since 1.0 · aow:Feedback

Definition. Structured information, from people or systems, used to improve future behavior.

Purpose. Records changes proposed as a result of Outcomes, reviews, and incidents, and who proposed them.

Attribute	Type	Required	Description
<code>feedback_type</code>	one of <code>accuracy</code> , <code>policy</code> , <code>exception</code> , <code>preference</code> , <code>incident</code>		What the Feedback is about.
<code>issue_detected</code>	text		What went wrong or could be better.
<code>recommended_adjustments</code>	string-list		Specific changes proposed.
<code>severity</code>	one of <code>low</code> , <code>medium</code> , <code>high</code> , <code>critical</code>		How urgent the Feedback is.
<code>timestamp</code>	datetime		When the Feedback was given.

Relationships.

- `feedback_on` → Outcome / Result / Action / Plan (one or more) (*required*)
- `provided_by` → Actor
- `updates` → Policy / Memory / Plan / Skill / Actor / Context (one or more)

4.23 Memory

Layer: Assurance · abstract · since 1.0 · aow:Memory

Definition. Persistent knowledge accumulated from experience, used to inform future Context.

Purpose. Retains information from past work for use in future Context and audit.

Attribute	Type	Required	Description
<code>store_ref</code>	uri		Where the memory is held.
<code>retention_policy</code>	text		How long entries are kept and when they are purged.

Notes.

- Two kinds: Operational Memory (short-lived) and Knowledge Base (long-lived).

4.24 Operational Memory

Layer: Assurance · a kind of Memory · since 1.0 · `aow:OperationalMemory`

Definition. Short-lived, execution-oriented memory scoped to a session, Task, or Work Item.

Purpose. Carries state across the steps of a single piece of work.

4.25 Knowledge Base

Layer: Assurance · a kind of Memory · since 1.0 · `aow:KnowledgeBase`

Definition. Long-lived memory of generalized patterns, rules, and precedents.

Purpose. Carries what was learned from one piece of work into the next.

5 Provenance and versioning

Version 1.0 listed "Provenance & Versioning" as an entity. In version 2.0, provenance is a set of attributes that any entity may carry. Every entity can have a version, an author, a creation time, and a previous version. A separate provenance entity would duplicate this information, which conflicts with the minimal redundancy criterion (section 13).

The provenance attributes use the W3C PROV Ontology (PROV-O):

Attribute	Meaning	RDF property
<code>version</code>	Version of this entity definition	<code>schema:version</code>
<code>created_at</code>	When this entity or version was created	<code>prov:generatedAtTime</code>
<code>created_by</code>	The Actor responsible for it	<code>prov:wasAttributedTo</code>
<code>revision_of</code>	The earlier version this one revises	<code>prov:wasRevisionOf</code>

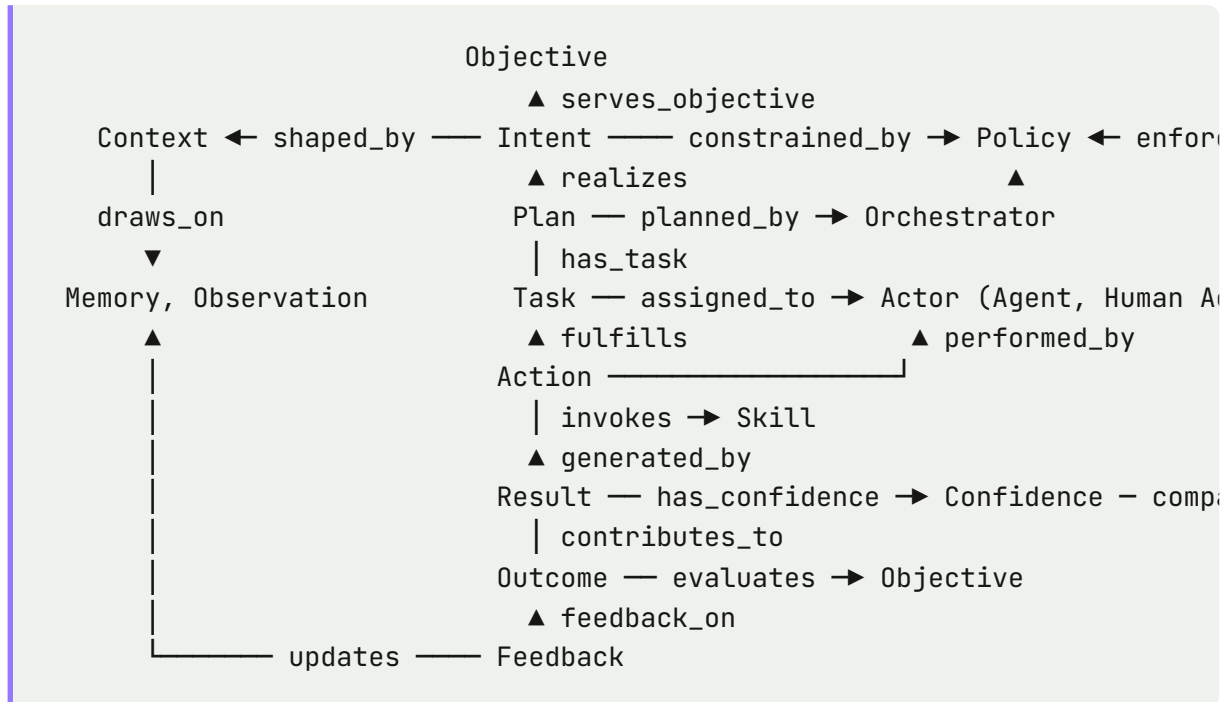
Several AOW classes and relationships are also defined as specializations of PROV-O terms, so tools that support PROV-O can read AOW data: Actor is a `prov:Agent`, Action is a `prov:Activity`, Result, Signal, and Observation are `prov:Entity`, Plan is a `prov:Plan`, `performed_by` specializes `prov:wasAssociatedWith`, `generated_by` specializes `prov:wasGeneratedBy`, and `derived_from` specializes `prov:wasDerivedFrom`.

For any entity, the provenance attributes record who created it, when, from what, and which version it revises. The explainability criterion in section 13 depends on this information.

6 The relationship model

6.1 The semantic graph

AOW is a directed graph that connects business goals to execution, and execution back to learning. The principal relationships are:



In words:

- An **Objective** gives rise to one or more **Intents**, each a specific, actionable part of the broader goal.
- Intents are **shaped by Context** and **constrained by Policy**, and state the **Assurance Level** they require.
- An **Orchestrator** turns an Intent into a **Plan** of **Tasks**, and assigns each Task to an **Actor**, chosen by Role, Skills, and autonomy.
- Actors carry out Tasks through **Actions**, each of which **invokes a Skill** and produces a **Result**.
- Results carry a **Confidence**, which is compared against the Assurance Level to decide whether a person must confirm.
- Results **contribute to Outcomes**, which are measured against the originating Objective.
- Outcomes (and people) generate **Feedback**, which **updates** Policies, Memory, Plans, and Skills.

- **Memory** is used to build future Context.
- The **Guardian** enforces Policy and Assurance Levels and escalates to people when required.

The website at <https://skandotai.github.io/agentive-ontology-of-work/> includes an interactive version of this graph.

6.2 All relationships

Relationship	From	To	Read backward as	Required on
serves_objective	Intent	Objective	produces intent	Intent
shaped_by	Intent, Task	Context (many)	shapes	
draws_on	Context	Memory, Observation (many)	informs	
concerns	Intent, Task, Action	Work Item (many)	is subject of	
constrained_by	Intent, Plan, Task, Actor, Skill, Role	Policy (many)	constrains	
requires_assurance	Intent, Task, Skill	Assurance Level	required by	Intent
realizes	Plan	Intent	realized by	Plan
planned_by	Plan	Actor	plans	
has_task	Plan	Task (many)	part of plan	
depends_on	Task	Task (many)	prerequisite of	
assigned_to	Task	Actor	assigned	
requires_role	Task	Role	required for	
fills_role	Actor	Role (many)	filled by	
has_skill	Actor	Skill (many)	available to	
performed_by	Action	Actor	performs	Action

Relationship	From	To	Read backward as	Required on
fulfills	Action	Task	fulfilled through	
invokes	Action	Skill	invoked by	
approved_by	Action, Plan, Result	Human Actor (many)	approves	
generated_by	Result, Signal, Observation	Action	generated	Result
derived_from	Observation	Signal (many)	source of	Observation
about	Observation	Work Item, Actor, Action, Task	observed by	
affects	Result	Work Item (many)	affected by	
has_confidence	Result, Observation, Action, Plan	Confidence	confidence of	
compared_against	Confidence	Assurance Level	threshold for	
contributes_to	Result	Outcome (many)	aggregates	
evaluates	Outcome	Objective	evaluated by	Outcome
feedback_on	Feedback	Outcome, Result, Action, Plan (many)	received feedback	Feedback
provided_by	Feedback	Actor	provided	
updates	Feedback	Policy, Memory, Plan, Skill, Actor, Context (many)	updated by	
writes_to	Actor	Memory (many)	written by	

Relationship	From	To	Read backward as	Required on
<code>enforces</code>	Guardian	Policy, Assurance Level (many)	enforced by	
<code>oversees</code>	Guardian	Actor, Plan, Skill (many)	overseen by	
<code>escalates_to</code>	Guardian, Policy	Role, Human Actor (many)	escalation target of	

6.3 Design notes

One direction per relationship. Each relationship is defined in one direction only. The direction is chosen so the entity that holds the information records the link. For example, a Result records the Action that generated it. The documentation gives an inverse reading for each relationship, but the ontology does not define separate inverse properties. Queries can traverse relationships in either direction.

Delegation. In version 1.0, an Orchestrator "delegates to" Agents. In version 2.0, delegation is recorded as a Plan produced by the Orchestrator (`planned_by`) that contains Tasks assigned to Actors (`assigned_to`). Each delegation is recorded once, on the Task.

Policies do not perform Actions. When a Policy blocks or escalates work, the block or escalation is recorded as an Action performed by a Guardian. The public-sector example in section 11 shows this.

7 Assurance Levels and governed autonomy

Version 1.0 used "AL2" and "Delegated" in its examples without defining a scale. Version 2.0 defines one.

7.1 The scale

An **Assurance Level** specifies the autonomy permitted for a piece of work and the human oversight required. Higher levels permit more autonomy and require more evidence before they are granted.

Level	Name	People	Agents	Default confidence threshold	Suited to
AL0	Manual	Performs the work.	Observes and prepares; takes no Action that changes a Work Item.	—	Any risk; the default for work not yet assessed.
AL1	Assisted	Decides. Approves every Action that changes a Work Item before it takes effect.	Recommends, drafts, and prepares.	—	High-risk or irreversible decisions; new or poorly characterized work.
AL2	Delegated	Confirms when confidence is low or a Policy triggers; reviews a sample otherwise.	Decides and acts within Policy.	0.90	Moderate-risk, well-characterized decisions.
AL3	Supervised	Reviews by exception, when the Guardian raises an alert, and in periodic audit.	Acts, and must be able to reverse or compensate its Actions.	0.95	Low-to-moderate risk, reversible effects.
AL4	Autonomous	Sets Policy and monitors aggregate Outcomes.	Acts without routine human review; the Guardian can suspend it at any time.	0.98	Low-risk, reversible, high-volume, well-characterized work.

The ontology defines the five levels as named individuals, `aow:AL0` to `aow:AL4`. Documents refer to a level by its identifier, for example `"requires_assurance": "aow:AL2"`.

The scale is based on research on levels of automation, including Sheridan and Verplank's ten-level scale (1978), Parasuraman, Sheridan, and Wickens's model of human interaction with automation (2000), and the SAE J3016 levels of driving automation. AOW uses five levels. Each level specifies what the person does, what the Agent may do, and what review is required.

7.2 The effective level

Four entities can set the Assurance Level for an Action:

- the **Intent** states the level its work requires (`requires_assurance` , required)
- a **Task** may lower that level for one step (`requires_assurance` , optional)
- a **Skill** may cap the level at which it may be used, typically because it has irreversible side effects
- an **Agent** has been cleared to operate up to a level (`autonomy_level`)

The **effective Assurance Level** of an Action is the lowest of these levels. For example, an Agent cleared for AL3 that works on an Intent requiring AL2 operates at AL2.

7.3 Confidence and escalation

Levels AL2, AL3, and AL4 have default confidence thresholds of 0.90, 0.95, and 0.98 (`default_confidence_floor`). If the Confidence of an Action that changes a Work Item is below the threshold, the Agent must escalate. The Agent records the Action with status `escalated` , and a person decides. A Policy may set a different threshold.

At AL0 and AL1, a person must approve every Action that changes a Work Item (`approved_by`). At AL3 and AL4, review takes place after the Action, so an Agent must not use a Skill with irreversible side effects without approval.

7.4 Verification

The query `queries/oversight-gaps.rq` implements these rules. It lists every Action by an Agent that changed a Work Item without the oversight required by its effective level. For conformant data, the query returns no results. The query runs against every example in continuous integration. The test fixtures include an invalid example, in which an Agent acts at 0.72 confidence under an AL2 Intent without approval, and the query detects it.

8 The lifecycle

The lifecycle describes one cycle of work, from business goal to feedback. Each stage lists the entities it creates or uses.

1. **Goal setting** (Objective). The business defines Objectives, tied to KPIs, SLAs, regulatory requirements, or operational targets. *Example: reduce claims cycle time by 20%.*
2. **Intent formation** (Intent). An Intent structures the Objective into machine-interpretable form and states the Assurance Level the work requires. *Example: automate eligibility determination, at AL2.*
3. **Perception** (Work Item, Signal, Observation). Signals about the Work Item are captured and interpreted into Observations. *Example: a claim form arrives; the claim amount is extracted.*
4. **Context enrichment** (Context, Memory). Real-time data, procedure, history, and Memory enrich the Intent. *Example: claim type disability; claimant age 45; policy form LTD-2024.*
5. **Policy application** (Policy). Policies filter what is permitted, required, or prohibited. *Example: auto-approve claims under \$50,000 when confidence is at least 0.90.*
6. **Planning** (Orchestrator, Plan, Task). The Orchestrator produces a Plan of Tasks: extract claim details, validate coverage, calculate the benefit, record the decision.
7. **Assignment** (Task, Actor, Role). Each Task is assigned to an Actor that can fill the required Role and holds the necessary Skills, at an autonomy level the Task allows.
8. **Execution** (Action, Skill). Actors perform Actions, each invoking a Skill, and record their rationale.
9. **Result generation** (Result, Confidence). Each Action produces a Result, annotated with its Confidence, artifacts, and the Work Items it affected.
10. **Assurance verification** (Guardian, Assurance Level, Human Actor). The Guardian checks compliance, Assurance Levels, Policies, and provenance, and escalates to people where required. People approve what their level requires them to.
11. **Outcome assessment** (Outcome). Results aggregate into Outcomes measured against the Objective: cycle time, error rate, SLA attainment.
12. **Feedback** (Feedback). Outcomes, reviewers, and incidents produce Feedback that updates Policies, Plans, Skills, and agent behavior.
13. **Memory update** (Memory, Knowledge Base). The trace is recorded for future Context, knowledge improvement, and audit.

The lifecycle includes all eleven stages from version 1.0. Perception (3) and Assignment (7) are new in version 2.0.

9 Formal representation

Version 1.0 stated that the ontology "should be implementable using JSON-LD, RDF/OWL, GraphQL schemas, and knowledge graph frameworks." Version 2.0 provides these implementations. All are generated from one source file, `aow.yaml` .

9.1 Namespace and identifiers

Namespace	<code>https://w3id.org/aow#</code> (prefix <code>aow:</code>)
Ontology IRI	<code>https://w3id.org/aow</code>
This version	<code>https://w3id.org/aow/2.0.0</code>
JSON-LD context	<code>https://w3id.org/aow/context.jsonld</code>

The identifiers use w3id.org, a permanent identifier service. They continue to resolve if the project's hosting changes.

9.2 The artifacts

Artifact	Format	What it is for
<code>aow.yaml</code>	YAML	The canonical source. Everything below is generated from it.
<code>ontology/aow.ttl</code> , <code>aow.jsonld</code>	OWL 2 (Turtle, JSON-LD)	The ontology itself, for reasoners, knowledge graphs, and ontology editors.
<code>ontology/context.jsonld</code>	JSON-LD 1.1 context	Lets plain AOW JSON be read as RDF with no changes.
<code>ontology/shapes.ttl</code>	SHACL	Checks that a graph of AOW data is well formed: types, required links, allowed values.
<code>schemas/*.schema.json</code>	JSON Schema 2020-12	Checks individual AOW JSON documents, for teams that never touch RDF.
<code>queries/*.rq</code>	SPARQL	Explanation and oversight checks (sections 7.4 and 10).
<code>examples/</code>	JSON-LD	The worked example and industry examples in this paper, all valid.

9.3 JSON and RDF

AOW documents are JSON, with `snake_case` keys, string identifiers, and a `type` for each entity. The JSON-LD context maps each key to an RDF property, so the same document can be read as RDF. The following Intent is from the worked example:

```

{
  "@context": ["https://w3id.org/aow/context.jsonld", { "@base": "https://",
  "@graph": [
    {
      "id": "intent-eligibility-2026",
      "type": "Intent",
      "goal_statement": "Determine eligibility for incoming disability cl
      "serves_objective": "obj-claims-cycle-time",
      "shaped_by": ["ctx-claims-procedure"],
      "constrained_by": ["policy-auto-approval", "policy-high-risk-review
      "requires_assurance": "aow:AL2",
      "priority": "high"
    }
  ]
}

```

Teams that use JSON validate the document against `schemas/intent.schema.json`. Teams that use a knowledge graph load it as RDF and validate it with the SHACL shapes. Both methods check the same rules.

The JSON Schemas reject unrecognized keys, such as the misspelling `goal_statment`. Keys with a namespace prefix, such as `acme:region`, are allowed for extensions (section 14).

9.4 The validation pipeline

Every example in the repository is validated against four checks in continuous integration:

1. **JSON Schema:** the document is well formed.
2. **SHACL:** the graph is well formed. For example, every Result has an Action, every Intent serves an Objective, and every relationship points to the correct type of entity.
3. **Explainability:** every Result can be traced to an Objective (`queries/untraced-results.rq` returns no results).
4. **Oversight:** no Agent acted without the oversight required by its effective Assurance Level (`queries/oversight-gaps.rq` returns no results).

The repository also contains invalid test documents. Each must fail a specific check.

9.5 A registry

A platform can store its AOW entities in a registry: a catalog of the Agents, Skills, Roles, Policies, and Assurance Levels in use, and the Intents, Plans, Tasks, and Actions that reference them. Any store that can hold AOW JSON documents can serve as a registry. A knowledge graph that loads them as RDF can also be queried. The `examples/` directory replaces the registry format in version 1.0, Appendix B, and contains complete registries for six scenarios.

10 Worked example: claims processing

This example extends the version 1.0 example to cover all four layers. It includes a claim that is escalated to a person. The complete document is `examples/claims-processing.jsonld`.

Objective. Reduce disability claims cycle time by 20% without increasing leakage.

Intent. Determine eligibility for incoming disability claims automatically wherever policy allows, and route the rest to an adjuster with the work already done. Requires **AL2**.

Policies. Auto-approve claims under \$50,000 when eligibility confidence is at least 0.90. Claims with fraud indicators, or from claimants over 65 with pre-existing conditions, require adjuster review.

Perception. A claim form arrives through the intake portal (a Signal). The Validation agent extracts the claim amount, \$38,200, with confidence 0.97 (an Observation about Work Item `claim-84933`).

Context. Claim type disability; claimant age 45; claim amount \$38,200; policy form LTD-2024.

Plan. The Claims orchestrator produces four Tasks, each depending on the last: extract claim metadata, validate coverage, determine the benefit, record the decision.

Actors and Skills. The Validation, Coverage, and Benefit agents fill the Eligibility processor Role and are each cleared for AL3. They use `extract_claim_fields`, `check_coverage`, `rule_validate`, `calculate_benefit`, and `record_decision`. Only `record_decision` changes a Work Item, and the change is reversible.

Claim 84933: decided by an agent. The decision Task requires AL2 and the Validation agent is cleared for AL3, so the effective level is AL2. The confidence threshold is 0.90. The agent's eligibility confidence is 0.93, so the agent records the decision and its rationale: *"Coverage active, benefit calculated, no high-risk indicators; claim under \$50,000 and eligibility confidence 0.93 meets the AL2 threshold of 0.90."*

Claim 84951: decided by a person. For this claim, the agent's confidence is 0.81, below the threshold. The agent records its Action as `escalated`, with a recommendation. A claims adjuster reviews the recommendation and the attending physician's statement and records the decision using the same `record_decision` Skill. The adjuster's Action has the same attributes as an agent's Action (Principle 7).

Outcome. Median turnaround falls from 48 hours to 7 minutes over the quarter, and 64% of claims are processed without manual handling. These figures are illustrative.

Feedback. The adjuster reports that claims from claimants over 65 with pre-existing conditions reach review only after a decision has been drafted. The Feedback updates the high-risk review Policy (now version 1.1.0, a revision_of 1.0.0) and the claims Knowledge Base.

Explaining a decision. The query `queries/explain-result.rq` traces each Result to its Objective:

Result	Action	Performed by	Skill	Task	Intent	Objective
Coverage active	act-coverage-84933	Coverage agent	check_coverage	Validate coverage	Automate eligibility determination	Faster disability claims
Monthly benefit \$3,180	act-benefit-84933	Benefit agent	calculate_benefit	Determine benefit	Automate eligibility determination	Faster disability claims
Eligibility approved (0.93)	act-decision-84933	Validation agent	record_decision	Record decision	Automate eligibility determination	Faster disability claims
Approved after review	act-review-84951	Claims adjuster 17	record_decision	Record decision	Automate eligibility determination	Faster disability claims

The table is produced from the recorded data. It shows auditors, regulators, and customers which Actor made each decision, using which Skill, for which Objective.

11 Industry examples

Each example below is a complete document in `examples/industry/`. Figures are illustrative.

Insurance: claims adjudication (AL2). Objective: reduce adjudication time. Intent: triage eligibility and approve low-risk claims automatically. Context: claimant age, policy details, historical exceptions. Policy: approve low-risk claims automatically; route others to an adjuster. Result: eligibility decision at 0.95 confidence. Outcome: improved SLA compliance. Feedback: extend auto-approval to small water-damage claims.

Banking: KYC and AML review (AL2). Objective: improve compliance throughput. Intent: verify identity documents and flag anomalies. Perception: a passport scan (Signal) yields a machine-readable-zone check (Observation). Policy: high-risk customers need human review of every verification. Result: documents verified. Outcome: faster onboarding. Feedback: adjust an anomaly threshold in the verification Skill.

Healthcare: prior authorization (AL1). Objective: reduce patient wait time. Intent: assess medical necessity and prepare a decision. Policy: advanced imaging needs clinical validation. Because the Intent requires AL1, the agent prepares the authorization and a clinical reviewer approves it before it takes effect (`approved_by`). Outcome: faster approvals. Feedback: update guidelines for repeat imaging.

Public sector: benefits determination (AL2). Objective: reduce backlog. Intent: check eligibility of routine applications. Policy: sensitive cases go to a specialist. When an application is flagged sensitive, the Guardian blocks the agent's Action and escalates; a specialist caseworker records the determination. All three Actions are recorded. Outcome: reduced backlog. Feedback: tune an income threshold.

Retail: supply chain exceptions (AL3). Objective: reduce stockouts. Intent: detect and resolve shipment anomalies. Perception: a carrier feed reports a 30-hour delay. Policy: auto-correct only when risk is under threshold and the correction can be reversed. The agent reroutes the shipment at 0.97 confidence, above the AL3 threshold of 0.95. An idempotency key prevents a retry from creating a duplicate booking. Outcome: fewer lost sales. Feedback: feed delay patterns into forecasting.

12 How AOW relates to existing work

AOW reuses or aligns with existing standards and research where they define the same concepts. This section describes each relationship and the differences.

12.1 Service-Oriented Architecture

SOA is AOW's closest precedent in purpose. The Open Group's SOA Ontology, later standardized as ISO/IEC 18384-3:2016, formalized services, contracts, and compositions so that architectures could be described consistently.

SOA concept	AOW equivalent	Key difference
Service	Agent	Agents reason, plan, and act within granted autonomy; services execute fixed operations.
Operation	Skill	Skills carry side effects, cost, and governance metadata.
Service contract	Skill schemas + Policy + Assurance Level	AOW adds confidence and risk to the contract.
Orchestration	Orchestrator and Plan	Plans are produced dynamically, under Context.
Registry (UDDI)	AOW registry (section 9.5)	Records learning and Outcomes, not only endpoints.

SOA describes the invocation of fixed operations. AOW describes execution that depends on context and is subject to graded autonomy.

A companion project, the [SOA-to-Agentic AI Terminology Mapping](#) (Skan, Inc., 2026), maps 28 SOA terms to agentic equivalents. `crosswalks/soa-to-agentic-terms.csv` maps each of those agentic terms to the corresponding AOW class or property.

12.2 BPMN and DMN

The Object Management Group's Business Process Model and Notation (BPMN) and Decision Model and Notation (DMN) both describe work. The main differences are:

- BPMN assumes flows are designed in advance; AOW supports Plans produced at run time, under Context.

- BPMN models tasks, not reasoning; AOW models the Objectives, Context, Policies, and Outcomes around the tasks.
- BPMN defines the sequence of work. AOW defines the meaning of each element of the work.

The standards can be used together. A BPMN process can be the source of an AOW Plan, and a DMN decision table can be the `rule_set` of an AOW Policy.

12.3 Multi-agent systems research

AOW's Agents act autonomously, its Skills correspond to the capabilities of multi-agent systems literature, and its Orchestrator is a coordination mechanism. Its cognitive vocabulary corresponds to the belief-desire-intention (BDI) model (Rao and Georgeff, 1995): Objectives play the part of desires, Intents of intentions, and Context and Memory of beliefs. The FIPA agent communication standards are an earlier attempt at interoperable agent semantics.

Multi-agent systems research rarely addresses enterprise governance. AOW adds Assurance Levels, Policies, the Guardian, provenance, and a defined treatment of Outcomes, Feedback, and Memory.

12.4 Knowledge graphs and W3C standards

AOW can be stored in a knowledge graph. The knowledge graph is the storage. AOW is the schema. Where the W3C has standardized a concept, AOW reuses it or aligns with it:

Standard	Used for
PROV-O	Provenance facet; Actor, Action, Result, Plan anchored in PROV classes (section 5)
Organization Ontology (ORG)	Role aligns with <code>org:Role</code>
SOSA/SSN	Observation aligns with <code>sosa:Observation</code>
ODRL 2.2	Policy aligns with <code>odrl:Policy</code> ; ODRL is one possible policy language
schema.org	Action aligns with <code>schema:Action</code>
OWL 2, SHACL, JSON-LD 1.1	The ontology's own formats

12.5 Process mining

Process mining reconstructs how work is performed from system event data (van der Aalst, 2016). AOW's Perception layer corresponds to its inputs. A Signal corresponds to an event in the IEEE 1849 (XES) event log standard. A Work Item corresponds to a case, or to an object in the Object-Centric Event Log (OCEL 2.0) format. An agent's Actions can be linked to the Signals they generate (`generated_by`). Agent work can then be analyzed and checked for conformance to its Plan with the same methods used for work performed by people and systems.

12.6 Agent protocols: MCP and A2A

MCP and A2A are open protocols for communication between agents and tools. AOW describes the work they carry:

- **Model Context Protocol (MCP)** exposes tools and resources to models. An MCP tool is an AOW Skill (its `binding` can name it), and an MCP resource is a source of Context.
- **Agent2Agent (A2A)** lets agents delegate work to one another. A2A's Agent Card describes an AOW Agent, its skills are AOW Skills, and its Tasks are AOW Tasks. AOW's Task states match A2A's task states, so a Task can be exchanged over A2A without conversion.

Both protocols are maintained under the Linux Foundation. Neither protocol records why work is done (Objective, Intent), which rules apply (Policy, Assurance Level), or what resulted (Outcome, Feedback). AOW records these.

12.7 Observability: OpenTelemetry

The OpenTelemetry semantic conventions for generative AI, in development at the time of writing, define spans for agent operations such as invoking an agent and executing a tool, and attributes that identify agents and tools. An `execute_tool` span can be recorded as an AOW Action, its trace as Signals, and its agent attributes as the performing Agent. AOW adds the Task, Intent, and Objective that the span served.

12.8 AI governance frameworks

AOW is not a compliance framework. It provides a structure for recording compliance evidence:

- **NIST AI Risk Management Framework (AI RMF 1.0).** The framework's four functions (Govern, Map, Measure, Manage) correspond to AOW's Policy and Assurance Levels, Context and Intents, Confidence and Outcomes, and Guardian and Feedback respectively.
- **ISO/IEC 42001:2023** (AI management systems) asks organizations to define roles, controls, and records for AI; AOW's Role, Policy, Guardian, and provenance facet are a vocabulary for those records.
- **EU AI Act (Regulation (EU) 2024/1689).** Article 12 requires high-risk systems to keep logs, and Article 14 requires effective human oversight. AOW's Action records and provenance attributes support the first requirement. Its Assurance Levels, `approved_by` relationship, and oversight check support the second. Compliance of a specific deployment is a legal determination outside the scope of AOW.

13 Validation criteria

Version 1.0 defined the criteria below. Version 2.0 adds the method used to check each one.

Criterion	What it requires	How AOW 2.0 checks it
Semantic completeness	Every necessary concept is represented, none is overloaded, each has a clear purpose.	Every class has a definition and a purpose (checked in CI). The six new entities close the gaps 1.0 left.
Minimal redundancy	No two entities serve the same role.	Explicit disjointness axioms (Result vs. Outcome, Signal vs. Observation, Task vs. Action, Skill vs. Action, Confidence vs. Assurance Level, Operational Memory vs. Knowledge Base, Agent vs. Human Actor). Provenance became a facet rather than a redundant entity.
Orthogonality	Entities serve independent functions.	Policies constrain; they never act (Guardian Actions do). Skills are capabilities; Actions are their use.
Extensibility	New entities can be added without breaking the structure.	Extension rules in section 14; namespaced extension keys are accepted by the JSON Schemas.
Formal mapability	Implementable in JSON-LD, RDF/OWL, and knowledge graph frameworks.	Published in OWL, JSON-LD, SHACL, and JSON Schema (section 9).
Governance-ready	Expresses versioning, provenance, auditability, risk tiering, and compliance constraints.	Provenance facet (PROV-O); Assurance Level scale; Policy; Guardian.
Execution-ready	Supports dynamic planning, multi-agent cooperation, and real-time context.	Plan, Task, and Orchestrator; Task states aligned with A2A; Context with validity.

Criterion	What it requires	How AOW 2.0 checks it
Explainability-ready	Intent to Action, Action to Result, and Result to Outcome must be reconstructable deterministically.	<code>queries/explain-result.rq</code> reconstructs the chain; <code>queries/untraced-results.rq</code> must return nothing for every example.

14 Extending AOW

AOW can be extended for specific industries and platforms. Extensions should follow these rules:

1. **Subclass rather than redefine.** A "Claims Agent" is an `aow:Agent` with extra attributes, not a new kind of actor.
2. **Use your own namespace.** Extension classes and properties live in your namespace, not `aow:`. In JSON, extension keys carry a prefix (`"acme:region": "EMEA"`), which the AOW JSON Schemas accept and the JSON-LD context lets you map.
3. **Align, then add.** Before adding a concept, check whether an AOW class plus a Policy, Context variable, or Skill attribute already expresses it.
4. **Propose what generalizes.** If an extension would be useful across organizations, propose it for the core ontology through the project's issue tracker.

Candidate extensions include Simulation (testing Plans before execution), Worklet (reusable Plan fragments), and Cost (accounting for the resources an Action consumes).

15 Limitations and open questions

AOW 2.0 has the following known limitations:

- **Multi-party work.** AOW assumes one organization's governance. It does not yet define how Policies, Assurance Levels, and trust apply when agents from different organizations collaborate.
- **Negotiation and delegation between agents.** AOW records that a Task was assigned. It does not model the negotiation that led to the assignment.
- **Identity and authorization.** AOW records who performed an Action. The credentials used are left to identity standards.
- **Calibration.** The confidence thresholds assume that Confidence values are calibrated. The `calibrated` attribute records whether a value has been calibrated.
- **Policy semantics.** AOW does not prescribe a policy language. It can verify that a Policy exists and was applied, but not the content of its rules.
- **Empirical validation.** The examples are illustrative. Models of production deployments, including cases AOW could not describe, are needed.

Comments on the modeling choices in this paper can be submitted as issues or pull requests at <https://github.com/Skandotai/agent-ontology-of-work>.

16 Glossary

Term	Definition
Action	A single execution event in which an Actor invokes a Skill, in service of a Task, at a point in time.
Actor	Anything that can be assigned Tasks, perform Actions, and be held to account for them: an Agent, a person, an Orchestrator, or a Guardian.
Agent	A software Actor that interprets Intents and Tasks and acts on them using Skills, within the autonomy it has been cleared for.
Assurance Level	A governance-set level of permitted autonomy, with the human oversight and evidence it requires.
Assurance Level scale	The five levels of permitted autonomy, AL0 (Manual) to AL4 (Autonomous).
Confidence	A model-, rule-, or human-derived measure of certainty about an Observation, a Result, or an Action.
Context	The structured situational information that gives relevance to an Intent.
Effective Assurance Level	The lowest of the levels set on the Intent, Task, and Skill, and the performing Agent's autonomy_level.
Feedback	Structured information, from people or systems, used to improve future behavior.
Guardian	The oversight Actor that enforces Policy and Assurance Levels, monitors behavior, and escalates to people when needed.
Human Actor	A person who performs, approves, reviews, or oversees work.
Intent	A structured, actionable goal derived from an Objective and interpretable by agents, orchestrators, and people.
Knowledge Base	Long-lived memory of generalized patterns, rules, and precedents.
Memory	Persistent knowledge accumulated from experience, used to inform future Context.
Objective	A business-level goal expressed in strategic or operational terms.

Term	Definition
Observation	An interpreted fact about a Work Item, Actor, or Action, derived from one or more Signals and carrying a Confidence.
Operational Memory	Short-lived, execution-oriented memory scoped to a session, Task, or Work Item.
Orchestrator	A coordinating Actor that turns Intents into Plans, assigns Tasks to Actors, and manages sequencing and exceptions.
Outcome	The aggregated business impact of one or more Results, measured against an Objective.
Plan	An ordered or partially ordered set of Tasks intended to fulfill an Intent, produced by an Orchestrator or other Actor.
Policy	Declarative constraints defining permissible, required, or restricted behavior.
Provenance facet	The attributes every entity may carry to record who created it, when, from what, and which version it revises, expressed with W3C PROV-O.
Result	The immediate, atomic effect or output of an Action.
Role	A named set of responsibilities and authority that an Actor can fill, such as "claims adjuster" or "eligibility reviewer".
Signal	A raw, time-stamped trace emitted by a system, a person, or the environment: an event, a log line, a user-interface interaction, a document arriving, a state change, a metric sample.
Skill	A reusable capability that an Actor can invoke.
Task	A unit of assigned work within a Plan, performed by an Actor filling a Role.
Work Item	The unit of business work that the rest of the ontology is about: a claim, an application, an order, a case, a ticket.

17 Summary

AOW is a platform-agnostic, formally specified ontology of how work performed by AI agents, people, and systems is structured, governed, executed, and improved. AOW 2.0 provides:

- separate definitions of Objective, Intent, Plan, Task, Action, Result, and Outcome
- a common Actor model for people and agents, with human approvals recorded
- governance within the model: Policy, the Assurance Level scale, confidence thresholds, and the Guardian
- a feedback cycle from Outcomes to Memory
- alignment with PROV-O, BPMN, MCP, A2A, OpenTelemetry, and other standards and protocols
- machine-readable files, with automated checks for traceability and oversight

AOW is proposed as a reference model for agentic enterprise work, and as input to future standards.

Appendix A: Relationship matrix

For each entity, the kinds of entity it can point to and be pointed to by (inherited relationships included).

Entity	Points to	Pointed to by
Work Item	—	Action, Intent, Observation, Result, Task
Signal	Action	Observation
Observation	Action, Actor, Confidence, Signal, Task, Work Item	Context
Objective	—	Intent, Outcome
Intent	Assurance Level, Context, Objective, Policy, Work Item	Plan
Context	Memory, Observation	Feedback, Intent, Task
Policy	Human Actor, Role	Actor, Feedback, Guardian, Intent, Plan, Role, Skill, Task
Plan	Actor, Confidence, Human Actor, Intent, Policy, Task	Feedback, Guardian
Task	Actor, Assurance Level, Context, Policy, Role, Task, Work Item	Action, Observation, Plan, Task
Agent	Memory, Policy, Role, Skill	Action, Feedback, Guardian, Observation, Plan, Task
Human Actor	Memory, Policy, Role, Skill	Action, Feedback, Guardian, Observation, Plan, Policy, Result, Task
Orchestrator	Memory, Policy, Role, Skill	Action, Feedback, Guardian, Observation, Plan, Task
Guardian	Actor, Assurance Level, Human Actor, Memory, Plan, Policy, Role, Skill	Action, Feedback, Guardian, Observation, Plan, Task
Role	Policy	Actor, Guardian, Policy, Task
Skill	Assurance Level, Policy	Action, Actor, Feedback, Guardian
Action	Actor, Confidence, Human Actor, Skill, Task, Work Item	Feedback, Observation, Result, Signal

Entity	Points to	Pointed to by
Result	Action, Confidence, Human Actor, Outcome, Work Item	Feedback
Confidence	Assurance Level	Action, Observation, Plan, Result
Assurance Level	—	Confidence, Guardian, Intent, Skill, Task
Outcome	Objective	Feedback, Result
Feedback	Action, Actor, Context, Memory, Outcome, Plan, Policy, Result, Skill	—
Operational Memory	—	Actor, Context, Feedback
Knowledge Base	—	Actor, Context, Feedback

Appendix B: Changes from version 1.0

Area	Version 1.0	Version 2.0
Entities	15 described, 17 in the registry	25 classes, 8 of them new: Work Item, Signal, Observation, Plan, Task, Role, Actor, Human Actor. Orchestrator and Guardian are now kinds of Actor
Action	In glossary and registry; no definition	Defined; produces Results
Result	Produced by Skill (section 3.8) or Action (glossary)	Produced by Action
Plan	"Optional intermediate concept"	Full entity; realizes an Intent
Task	Used throughout; undefined	Defined; states follow A2A
People	Implicit ("human review")	Human Actor and Role; <code>approved_by</code>
Autonomy	"AL2," "Delegated" used, undefined	AL0 to AL4 scale; effective-level rule; confidence floors
Provenance & Versioning	Two registry entities without attributes	A facet of every entity, expressed with PROV-O
Guardian	Definition only	Actor with attributes and relationships; its interventions are Actions
Memory	Definition and two types	Abstract class with two subclasses and attributes
Assurance Level	No relationships	Referenced by Intent, Task, Skill, Agent, Confidence, Guardian
Skill	No side-effect information	<code>side_effects</code> : none, reversible, irreversible
Identifiers	<code>intent_id</code> , <code>agent_id</code> , and so on	<code>id</code> on every entity; <code>type</code> names the class

Area	Version 1.0	Version 2.0
Agent	<code>skills</code> , <code>memory_link</code> , <code>role</code> , <code>assurance_level_required</code> attributes	<code>has_skill</code> , <code>writes_to</code> , <code>fills_role</code> relationships; required assurance moved to Intent, Task, and Skill
Machine-readable form	Described	OWL, JSON-LD, SHACL, JSON Schema, SPARQL, validated examples
Validation criteria	Stated	Each paired with a check
Relation to other work	SOA, BPMN, MAS, knowledge graphs	Adds PROV-O and W3C standards, process mining, MCP, A2A, OpenTelemetry, NIST AI RMF, ISO/IEC 42001, EU AI Act
Editorial	Mixed section numbering; version referred to as "v3" in section 11; example YAML lost its indentation	Corrected throughout

References

- Berti, A., et al. (2024). *OCEL (Object-Centric Event Log) 2.0 Specification*. arXiv:2403.01975.
- Erl, T. (2005). *Service-Oriented Architecture: Concepts, Technology, and Design*. Prentice Hall.
- European Union (2024). Regulation (EU) 2024/1689 (Artificial Intelligence Act). *Official Journal of the European Union*.
- FIPA (2002). *FIPA ACL Message Structure Specification* (SC00061G). Foundation for Intelligent Physical Agents.
- Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2), 199–220.
- IEEE (2016). *IEEE Standard for eXtensible Event Stream (XES) for Achieving Interoperability in Event Logs and Event Streams* (IEEE 1849).
- ISO/IEC (2016). *ISO/IEC 18384-3:2016 Information technology — Reference Architecture for Service Oriented Architecture (SOA RA) — Part 3: Service Ontology*.
- ISO/IEC (2023). *ISO/IEC 42001:2023 Information technology — Artificial intelligence — Management system*.
- Linux Foundation. *Agent2Agent (A2A) Protocol Specification*. <https://a2a-protocol.org>
- Model Context Protocol. *Model Context Protocol Specification*. <https://modelcontextprotocol.io>
- NIST (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)* (NIST AI 100-1). National Institute of Standards and Technology.
- Object Management Group (2013). *Business Process Model and Notation (BPMN), Version 2.0.2*.
- Object Management Group. *Decision Model and Notation (DMN)*.
- OpenTelemetry. *Semantic Conventions for Generative AI Systems*. <https://opentelemetry.io/docs/specs/semconv/gen-ai/>
- Parasuraman, R., Sheridan, T. B., and Wickens, C. D. (2000). A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics — Part A*, 30(3), 286–297.
- Rao, A. S., and Georgeff, M. P. (1995). BDI agents: From theory to practice. *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*, 312–319.
- SAE International (2021). *J3016: Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles*.

- Sheridan, T. B., and Verplank, W. L. (1978). *Human and Computer Control of Undersea Teleoperators*. MIT Man-Machine Systems Laboratory.
- Skan, Inc. (2026). *SOA-to-Agentive AI Terminology Mapping*, version 2.0.0. Zenodo. <https://doi.org/10.5281/zenodo.21823088>
- The Open Group. *Service-Oriented Architecture Ontology*, Technical Standard.
- van der Aalst, W. M. P. (2016). *Process Mining: Data Science in Action* (2nd ed.). Springer.
- W3C (2012). *OWL 2 Web Ontology Language Document Overview (Second Edition)*. W3C Recommendation.
- W3C (2013). *PROV-O: The PROV Ontology*. W3C Recommendation.
- W3C (2014). *The Organization Ontology*. W3C Recommendation.
- W3C (2017). *Shapes Constraint Language (SHACL)*. W3C Recommendation.
- W3C (2017). *Semantic Sensor Network Ontology*. W3C Recommendation.
- W3C (2018). *ODRL Information Model 2.2*. W3C Recommendation.
- W3C (2020). *JSON-LD 1.1*. W3C Recommendation.
- Wooldridge, M. (2009). *An Introduction to MultiAgent Systems* (2nd ed.). Wiley.